

NAME;S.Harshavardhan

Reg;192524106

Course;artificial intelligence

Aim

To implement the **8-Queen Problem** using the **Backtracking algorithm** in Python and place eight queens on an 8×8 chessboard so that no two queens attack each other.

Procedure

1. Start with an empty 8×8 chessboard.
2. Place a queen in the first column at a safe position.
3. Check that no queen exists in the same row, upper-left diagonal, or lower-left diagonal.
4. If the position is safe, place the queen and move to the next column.
5. If no safe position is available, backtrack to the previous column and try another position.
6. Repeat the process until all eight queens are placed successfully.
7. Display the final arrangement of the queens.

Algorithm – 8-Queen Problem

1. Start.
2. Create an empty 8 × 8 chessboard.
3. Begin placing queens from the first column.
4. Check if the current position is safe:
 - No queen in the same row.
 - No queen in the upper-left diagonal.
 - No queen in the lower-left diagonal.
5. If the position is safe, place the queen.
6. Move to the next column and repeat the process.
7. If no safe position is found, backtrack to the previous column and try another position.
8. Continue until all 8 queens are placed.
9. Display the final chessboard.

10. Stop.

Pseudocode – 8-Queen Problem

BEGIN

Create an empty 8×8 board

FUNCTION Solve(Column)

 IF Column = 8 THEN

 RETURN TRUE

 FOR each Row from 0 to 7

 IF Position(Row, Column) is Safe THEN

 Place Queen

 IF Solve(Column + 1) = TRUE THEN

 RETURN TRUE

 ENDIF

 Remove Queen

 ENDIF

ENDFOR

RETURN FALSE

END FUNCTION

Call Solve(0)

Display the board

Observation

The Backtracking algorithm successfully places eight queens on the chessboard such that no two queens attack each other. Whenever a conflict occurs, the algorithm backtracks and tries another position until a valid solution is found.

Result

The **8-Queen Problem** was successfully implemented in Python using the **Backtracking algorithm**, and a valid arrangement of eight queens was obtained.

The screenshot displays a Python IDE with a dark theme. On the left, a code editor shows the implementation of the 8-Queen problem using backtracking. The code defines an 8x8 board, a helper function `isSafe` to check for conflicts, and a recursive `solve` function. The `solve(0)` function is called at the bottom, which prints the solution. On the right, the IDE's output panel shows the execution status as 'Successfully executed' with a time of 0.0100 secs and memory usage of 8.8 Mb. Below this, the 'Your Output' section displays the solution as an 8x8 grid of 0s and 1s, where each row contains exactly one 1, representing the positions of the eight queens.

```
N = 8
board = [[0 for i in range(N)] for j in range(N)]

def isSafe(row, col):
    for i in range(col):
        if board[row][i] == 1:
            return False

    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1

    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
        i += 1
        j -= 1

    return True

def solve(col):
    if col >= N:
        return True

    for i in range(N):
        if isSafe(i, col):
            board[i][col] = 1

            if solve(col + 1):
                return True

            board[i][col] = 0

    return False

if solve(0):
    print("Solution:")
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time: 0.0100 secs Memory: 8.8 Mb

Your Output

Solution:

```
[1, 0, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 1, 0]
[0, 0, 0, 0, 1, 0, 0, 0]
[0, 0, 0, 0, 0, 0, 0, 1]
[0, 1, 0, 0, 0, 0, 0, 0]
[0, 0, 0, 1, 0, 0, 0, 0]
[0, 0, 0, 0, 0, 1, 0, 0]
[0, 0, 1, 0, 0, 0, 0, 0]
```

NAME;S.Harshavardhan

REG;192524106

Course;AI

Aim

To implement the **8-Puzzle Problem** using the **Breadth-First Search (BFS)** algorithm in Python to find the shortest path from the initial state to the goal state.

Procedure

1. Start the program.
2. Define the initial state and the goal state of the 8-puzzle.
3. Create a queue and insert the initial state.
4. Remove the first state from the queue.
5. Check whether the current state is the goal state.
6. If the goal state is reached, display the solution and stop.
7. Otherwise, generate all possible moves by moving the blank tile (up, down, left, or right).
8. Add the new valid states to the queue if they have not been visited.
9. Repeat the above steps until the goal state is found.
10. Display the final solution and stop.

Algorithm – 8-Puzzle Problem (Breadth-First Search)

1. Start.
2. Define the initial state and the goal state.
3. Insert the initial state into the queue.
4. Remove the first state from the queue.
5. Check whether it is the goal state.
6. If it is the goal state, display the solution and stop.
7. Otherwise, generate all possible next states by moving the blank tile (up, down, left, or right).
8. Add the new states to the queue if they have not been visited.

9. Repeat Steps 4–8 until the goal state is found or the queue becomes empty.
10. Stop.

Pseudocode – 8-Puzzle Problem

BEGIN

Initialize Queue

Insert Initial State into Queue

Mark Initial State as Visited

WHILE Queue is not Empty DO

 Remove First State from Queue

 IF Current State = Goal State THEN

 Display Solution

 STOP

 ENDIF

 Generate all Valid Moves

 FOR each New State DO

 IF New State is not Visited THEN

 Add New State to Queue

 Mark New State as Visited

 ENDIF

 ENDFOR

ENDWHILE

Display "No Solution Found"

END

Observation

The BFS algorithm explores all possible states level by level and finds the shortest sequence of moves to reach the goal state. The puzzle is solved successfully when the blank tile is moved to the correct position.

Result

The **8-Puzzle Problem** was successfully implemented in Python using the **Breadth-First Search (BFS)** algorithm, and the goal state was reached successfully.

```
1 # 8 Puzzle Problem
2
3 initial = [1, 2, 3,
4           4, 5, 6,
5           0, 7, 8]
6
7 goal = [1, 2, 3,
8         4, 5, 6,
9         7, 8, 0]
10
11 print("Initial State:")
12 print(initial[0:3])
13 print(initial[3:6])
14 print(initial[6:9])
15
16 print("\nGoal State:")
17 print(goal[0:3])
18 print(goal[3:6])
19 print(goal[6:9])
20
21 print("\nMove Blank Right")
22 initial[6], initial[7] = initial[7], initial[6]
23
24 print(initial[0:3])
25 print(initial[3:6])
26 print(initial[6:9])
27
28 print("\nMove Blank Right Again")
29 initial[7], initial[8] = initial[8], initial[7]
30
31 print(initial[0:3])
32 print(initial[3:6])
33 print(initial[6:9])
34
35 print("\nGoal Reached")
36
```

Enter Input here

If your code takes input, add it in the above box before running.

Output

Status : Successfully executed

Time:	Memory:
0.0100 secs	8.672 Mb

Your Output

Initial State:

```
[1, 2, 3]
[4, 5, 6]
[0, 7, 8]
```

Goal State:

```
[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```